

RN STUDIO

Complete Phase-by-Phase Build Plan

An AI-Reasoning React Native Developer-Observability Platform

Prepared for: **Abhyodaya Pratap Singh**

Format: this document mirrors a phase-execution spec — each phase has an Objective, Tasks, an Acceptance Test that gates moving on, and an Antigravity prompt you can paste directly into Google Antigravity (Gemini) to execute that phase.

Build target: full scope, no cut corners — SDK, cloud ingestion, Redis/BullMQ, multi-tenant auth, WebSocket live updates, retention, and a dedicated AI reasoning layer.

Timeline: ~33 focused working days (about 4.5-5 weeks) at 8h/day through Phase 13 (public v1.0). Stretch phases 14-15 are optional, post-launch.

0. Where You Actually Are

Nothing has been written yet — this plan assumes a blank repository starting today. Every phase below depends only on the phases before it. Treat this single document as the authoritative plan: do not start a phase until the previous phase's acceptance test passes. If a phase runs long, cut scope inside that phase — never skip its acceptance test to move on. This is the same discipline that keeps a hand-rolled distributed system correct, and it applies just as much to an ingestion pipeline: skipping a verification step to "move faster" is exactly how silent data loss or a broken auth boundary gets shipped without anyone noticing.

How to use this with Antigravity

- Open one phase at a time. Paste that phase's Objective + Tasks + Acceptance Test + Antigravity Prompt into the agent as one message — don't paste the whole document at once, it produces shallow, unfocused output.
- After the agent finishes a phase, run the acceptance test yourself before telling it to move on. The agent should not self-certify a phase as done.
- When a phase references an earlier phase's schema or contract (e.g. Phase 4 referencing the StudioEvent type from Phase 1), paste that specific snippet alongside the prompt so the agent has the exact contract, not a paraphrase.
- Keep each phase in its own conversation/session in Antigravity where possible — long single-session context tends to drift on a multi-week project like this.

1. Tech Stack (Confirmed, Unchanged Across All Phases)

Layer	Technology	Why
Mobile SDK	React Native + TypeScript	Core product; embeds into any RN app
Dashboard	Next.js + TypeScript + Tailwind	Session views, live tab, AI insights UI
API	Node.js + Fastify	Ingestion, auth, project management
Realtime	ws / Socket.IO	Live session streaming to dashboard
Queue	Redis + BullMQ (Upstash)	Async event processing, retries, rate limits
Database	PostgreSQL (Neon)	Relational store for projects/sessions/events
ORM	Prisma	Typed schema + migrations
Auth	JWT (dashboard) + hashed API keys (SDK)	Two separate, never-shared auth paths
AI layer	Claude API (Sonnet)	Summarization, anomaly detection, NL query, fix suggestions
Validation	Zod	Runtime validation at every boundary
Observability	OpenTelemetry	Traces/metrics for the platform itself
Testing	Vitest, React Native Testing Library, Playwright	Unit, component, E2E
Storage	Cloudflare R2	Session exports, large payloads
Infra	Docker + GitHub Actions	Local parity + CI/CD
Package publishing	npm	Real SDK distribution

Phase 0 — Project Definition & Repository Setup (~1 day)

Objective

Turn the idea into a measurable, bounded MVP instead of an open-ended feature list, and get a monorepo running so every later phase has somewhere to land.

Tasks

- Write the one-sentence product promise: "Install one SDK and inspect a running React Native app from inside the app and from a web dashboard, with an AI layer that explains what happened."
- Define the four core signal types for v1: network, console/logs, navigation, device/runtime info.
- Write the explicit non-goals list: no crash symbolication, no full video replay, no arbitrary JS execution from the dashboard, no production telemetry collection by default (opt-in only).
- Create the pnpm-workspace monorepo: apps/dashboard, apps/api, apps/worker, apps/demo-rn, packages/sdk, packages/protocol, packages/config.
- Create the demo RN app that becomes the permanent integration-test target for every later phase.
- Create docs/ADR/ (Architecture Decision Records) and write ADR-001 documenting the monorepo/package boundary decision.

Acceptance Test

A fresh clone of the repo can run **pnpm install** and start the demo RN app plus a placeholder dashboard/API locally, with no errors. ADR-001 exists and explains the package boundaries.

Antigravity Prompt for This Phase

```
Initialize a pnpm-workspace monorepo named rn-studio with apps/dashboard (Next.js + TS + Tailwind), apps/api (Fastify + TS), apps/worker (empty TS package for now), apps/demo-rn (React Native + TS, bare template), packages/sdk (empty TS package), packages/protocol (empty TS package), packages/config (shared eslint/tsconfig). Add root package.json, pnpm-workspace.yaml, a root tsconfig.base.json, and a docs/ADR/ folder with ADR-001-monorepo-structure.md documenting why these package boundaries were chosen. Verify `pnpm install` succeeds and the demo RN app and dashboard both start locally.
```

Phase 1 — SDK Core — Event Bus, Ring Buffer, Shared Types (~2 days)

Objective

Build the internal plumbing every collector and every UI panel will depend on, before any UI or network code exists. This is the equivalent of Concord's Phase 1 core state machine — invisible, but everything later calls into it.

Tasks

- Define the StudioEvent discriminated union in packages/protocol: network, console, navigation, device variants.
- Implement a tiny pub/sub event bus in packages/sdk: studio.emit(event) / studio.subscribe(listener).
- Implement RingBuffer with a hard size ceiling (default 500) — push() evicts oldest on overflow, getAll() returns a snapshot.
- Implement a redaction utility: a pure function that strips known-sensitive keys (authorization, cookie, password, token) from an arbitrary object before it is ever emitted onto the bus.

- Write unit tests (Vitest) for the ring buffer eviction behavior, the event bus subscribe/unsubscribe lifecycle, and the redaction function against a table of sensitive-key fixtures.

Acceptance Test

Unit tests pass for: ring buffer never exceeds its configured limit and evicts oldest-first; a subscriber receives every emitted event in order; the redaction function removes every known-sensitive key from a nested fixture object and leaves everything else untouched.

Antigravity Prompt for This Phase

In packages/protocol, define a StudioEvent discriminated union (type: 'network' | 'console' | 'navigation' | 'device', with a typed payload per variant). In packages/sdk, implement: (1) a minimal pub/sub EventBus class with emit/subscribe/unsubscribe, (2) a generic RingBuffer<T> class with a constructor limit (default 500), push() that evicts oldest on overflow, and getAll() returning a copy, (3) a redact(obj) pure function stripping keys named authorization, cookie, password, and token (case-insensitive, recursively) from an arbitrary nested object without mutating the input. Write Vitest unit tests covering ring buffer eviction, event bus ordering, and redaction against nested fixtures with sensitive keys at multiple depths.

Phase 2 — Draggable DevTools Panel (In-App UI Shell) (~2 days)

Objective

Build the visually memorable part of the product first — the thing a recruiter or interviewer actually sees in the README GIF — as an empty shell wired to the Phase 1 event bus, before any real data flows into it.

Tasks

- Build RNStudio.tsx: the public SDK entry component, rendered conditionally behind `__DEV__` in the consuming app.
- Build a draggable floating bubble using React Native's gesture/animation APIs; persist its last position locally (in-memory is fine for v1, AsyncStorage optional).
- Build the expandable panel shell with four empty tabs: Network / Logs / Navigation / Device.
- Isolate the panel's rendering from the host app's navigation tree (render via a portal-like top-level mount, not inside app screens) so it never interferes with the app's own UI.
- Feed the panel a fake/mock event stream in dev to verify the tab UI renders lists correctly before real collectors exist.

Acceptance Test

Inside the demo RN app, `<RNStudio />` renders a draggable bubble; dragging it and releasing keeps it at the drop position; tapping it opens a panel with four tabs that switch correctly and render a mocked list of items without crashing or blocking touches on the underlying app screen.

Antigravity Prompt for This Phase

Build `packages/sdk/src/RNStudio.tsx` as the public entry component, plus `overlay/DebugBubble.tsx` (draggable, using `react-native-gesture-handler` + `reanimated` or the RN `PanResponder` API, persisting last drop position in component state) and `overlay/DebugPanel.tsx` (expandable panel with four tabs: Network, Logs, Navigation, Device). Mount the panel independent of the host app's navigation stack so it renders above all screens without intercepting touches meant for the app. Feed each tab a small mocked array of fake events for now so the list UI can be visually verified before real collectors exist. Wire it into `apps/demo-rn/App.tsx` behind a `__DEV__` check.

Phase 3 — Collectors — Network, Console, Navigation, Device (~3 days)

Objective

Replace every mocked tab from Phase 2 with real, live data, captured safely (redacted before it ever reaches the ring buffer).

Tasks

- Network collector: wrap `global.fetch`, capture `method/url/status/duration/requestId` per call, run every captured object through the Phase 1 `redact()` before emitting.
- Console collector: wrap `console.log/warn/error/info` in dev, preserve original console behavior, emit a `console StudioEvent` per call.
- Navigation collector: build a React Navigation state-change listener adapter that emits `from/to/params` (params redacted) on every route change.
- Device collector: capture app version, RN version, platform, OS version, screen dimensions, and a generated installation/session ID once at panel mount.
- Wire all four collectors into the Phase 1 `RingBuffer` and the Phase 2 panel tabs; add basic search/filter input per tab.
- Add an axios adapter as a secondary network interceptor (optional at this phase, can move to Phase 3.5 if time is tight).

Acceptance Test

Running the demo RN app and triggering a real network request, a `console.log` call, a screen navigation, and app launch each produce a correctly-typed, redacted event visible in the matching panel tab within one render frame. Searching/filtering a tab narrows the visible list correctly.

Antigravity Prompt for This Phase

```
Implement four collectors in packages/sdk/src/collectors/: network.ts (wraps global.fetch, captures method/url/status/duration/requestId, redacts headers/body via the Phase 1 redact() before emit), console.ts (wraps console.log/warn/error/info while preserving original behavior), navigation.ts (a React Navigation NavigationContainer onStateChange adapter emitting from/to/params with params redacted), and device.ts (captures Platform.OS, Platform.Version, app version, screen dimensions, and a generated UUID session ID once). Wire all four into the existing EventBus/RingBuffer from Phase 1 and replace the mocked data in Phase 2's panel tabs with the live ring buffer contents. Add a simple text-filter input above each tab's list.
```

Phase 4 — Cloud Ingestion API & Multi-Tenant Auth (~3 days)

Objective

Give the SDK somewhere real to send data, with project-scoped authentication enforced on every route from day one — not bolted on later. This mirrors why Concord builds persistence's correctness rules before adding more features: auth boundaries are foundational, not decoration.

Tasks

- Define the `Project/User/ProjectMember/Session/Event` Prisma models (see Phase 6 for the full schema — stub the API against it now).

- Implement API-key auth for SDK ingestion: generate once, show once, store only the hash; a `resolveProject` middleware looks up the project from the key hash on every ingestion route.
- Implement JWT auth for the future dashboard: signup/login endpoints, short-lived access token + refresh token, project membership resolved from the token's user id — build this now even though the dashboard UI comes in Phase 7.
- Build the ingestion routes: `POST /v1/sessions`, `POST /v1/sessions/:id/events`, `POST /v1/sessions/:id/end`, with Zod validation on every payload.
- Add per-project rate limiting (in-memory counter is acceptable as a placeholder until Phase 5's Redis pipeline replaces it).
- Write integration tests: a request with no key is rejected, a request with another project's key cannot read/write this project's session, a malformed payload is rejected with a structured error envelope.

Acceptance Test

A valid API key can create a session and post a batch of events; an invalid or missing key returns 401; a key belonging to a different project cannot access another project's session (403, tested explicitly, not assumed); an oversized or malformed event batch is rejected with the documented error envelope shape, not a 500.

Antigravity Prompt for This Phase

In `apps/api` (Fastify), implement: (1) Prisma schema stubs for `Project`, `User`, `ProjectMember`, `Session`, `Event` per the schema in Phase 6 of this plan, (2) API-key auth middleware – API keys are generated once, shown once, stored only as a `bcrypt/argon2` hash, and a `resolveProject(req)` function resolves the acting project from the key's hash, (3) JWT auth for future dashboard use – signup/login routes issuing a short-lived access token and a refresh token, with project membership resolved via `ProjectMember`, (4) ingestion routes `POST /v1/sessions`, `POST /v1/sessions/:id/events`, `POST /v1/sessions/:id/end`, each Zod-validated, (5) a simple in-memory per-project rate limiter as a placeholder. Use the error envelope `{ error: { code, message, requestId } }` for every rejection. Write integration tests proving cross-project access is rejected with 403 and malformed payloads are rejected with 400, not 500.

Phase 5 — Redis / BullMQ Pipeline & Worker (~2 days)

Objective

Stop writing events synchronously into Postgres on the request path. Introduce the same async, retry-safe pipeline a production ingestion system needs.

Tasks

- Provision Upstash Redis; wire BullMQ with three queues: ingestion, ai-jobs (stubbed until Phase 10), retention (stubbed until Phase 9).
- API's POST /v1/sessions/:id/events now enqueues the validated batch onto the ingestion queue instead of writing directly.
- Build apps/worker: a BullMQ consumer for the ingestion queue that validates, normalizes, and batch-inserts events into Postgres via Prisma.
- Add exponential-backoff retry (max 5 attempts) and a dead-letter queue for batches that fail validation repeatedly.
- Replace Phase 4's in-memory rate limiter with a Redis-backed token-bucket counter keyed by API key.
- Add OpenTelemetry counters for queue depth and job processing duration.

Acceptance Test

Posting an event batch returns immediately (job enqueued, not written yet); within a few seconds the events are visible in Postgres; a batch engineered to fail validation is retried up to 5 times with backoff and then lands in the dead-letter queue, visible via a debug endpoint, not silently dropped.

Antigravity Prompt for This Phase

```
Wire BullMQ (ioredis client against Upstash Redis) with three queues: ingestion, ai-jobs, retention (the latter two can be created empty for now). Change apps/api's POST /v1/sessions/:id/events to enqueue the Zod-validated batch onto the ingestion queue and return 202 immediately, rather than writing to Postgres synchronously. Build apps/worker as a BullMQ consumer for the ingestion queue: normalize each event, batch-insert via Prisma, use exponential backoff with a 5-attempt cap, and route permanently-failing jobs to a dead-letter queue with a debug endpoint to inspect it. Replace the Phase 4 in-memory rate limiter with a Redis-backed token-bucket keyed by API key. Add OpenTelemetry counters for ingestion queue depth and worker job duration.
```

Phase 6 — PostgreSQL Schema & Persistence (~2 days)

Objective

Finalize the relational model the whole platform reads and writes against, with the indexes the actual query patterns need.

Tasks

- Finalize Prisma schema: Project(id, name, apiKeyHash, retentionDays, createdAt), User, ProjectMember(userId, projectId, role), Session(id, projectId, platform, appVersion, startedAt, endedAt, aiSummary), Event(id, sessionId, type, timestamp, payload Json), Finding (for Phase 10's anomaly flags), RetentionAudit (for Phase 9).
- Add indexes: Session(projectId, startedAt), Event(sessionId, timestamp), Event(type, timestamp).

- Write and run the first real migration; review the generated SQL by hand before applying.
- Build a small repository layer (packages or apps/api/src/repositories) so raw Prisma calls aren't scattered through route handlers.
- Write tests asserting the query patterns used by the (upcoming) dashboard hit the indexes above (EXPLAIN-based or at minimum a query-shape test).

Acceptance Test

prisma migrate deploy runs cleanly against a fresh database; a session with 10,000 events can be paginated by the dashboard's expected query shape in under 100ms locally; EXPLAIN on the main session-list and event-list queries shows index usage, not a sequential scan.

Antigravity Prompt for This Phase

Finalize the Prisma schema per Section 6 of this plan (Project, User, ProjectMember, Session, Event, Finding, RetentionAudit) with the indexes listed. Generate and review the first migration by hand. Build a repository layer (e.g. apps/api/src/repositories/sessionRepository.ts, eventRepository.ts) wrapping the common query shapes: paginated session list by project, cursor-paginated event list by session. Write a script or test that runs EXPLAIN on both queries against a seeded database with 10,000+ events and asserts an index scan is used, not a sequential scan.

Phase 7 — Dashboard Core — Sessions List, Session Detail (~3 days)

Objective

Build the website half of the product: the place a developer actually goes to read what happened, reading purely from the Phase 6 schema via the Phase 4 JWT auth.

Tasks

- Auth-gated Next.js layout: login page, project selector, protected routes.
- Sessions list page (/projects/:id/sessions): platform, app version, duration, event count, paginated.
- Session detail page (/sessions/:id): Network table, Console/Logs list, Navigation timeline, Device info panel — one tab per signal, mirroring the in-app panel's structure so the mental model is consistent across surfaces.
- Request detail drawer: timing, redacted headers, optional body toggle, copy-as-cURL.
- Loading/empty/error states for every list and detail view — do not ship bare spinners with no empty-state message.

Acceptance Test

Logging in as a project member shows only that project's sessions; opening a session shows all four signal tabs populated from real ingested data; pagination on a session with 10,000+ events does not load them all at once; every list/detail view has a distinguishable empty state versus a loading state.

Antigravity Prompt for This Phase

Build the Next.js dashboard core: an auth-gated layout (login, project selector) wired to the Phase 4 JWT endpoints, `apps/dashboard/app/projects/[id]/sessions/page.tsx` (paginated sessions list – platform, version, duration, event count), and `apps/dashboard/app/sessions/[id]/page.tsx` with four tabs (Network, Logs, Navigation, Device) reading from the Phase 6 repository layer's endpoints. Add a request detail drawer for the Network tab showing timing, redacted headers, an optional body toggle, and a copy-as-cURL button. Add explicit loading, empty, and error states to every list and detail component – no bare spinners with no accompanying message.

Phase 8 — WebSocket Live Updates (~2 days)

Objective

Let a developer watch a session as it happens, not just after it ends — this is the feature that turns the dashboard from a log viewer into a live debugging tool.

Tasks

- Build a WebSocket gateway service, authenticated using the same JWT as the REST dashboard API, scoped per project+session channel.
- After the Phase 5 worker successfully writes a batch, publish a lightweight notification to a Redis pub/sub channel for that session.
- WS gateway subscribes to Redis pub/sub and fans out to connected dashboard clients watching that session.
- Build `/sessions/:id/live` in the dashboard: subscribes on mount, renders incoming events into the same tab components from Phase 7 with a visible 'live' indicator, unsubscribes cleanly on unmount.

- Client-side reconnect with exponential backoff; a duplicate-subscription guard so a reconnect doesn't create two active listeners on the same channel.

Acceptance Test

Opening `/sessions/:id/live` while the demo app is actively sending events shows new rows appearing in the Network/Logs/Navigation tabs within about one second of the real event, without a page refresh. Killing and restoring network connectivity to the dashboard tab reconnects automatically without duplicating already-shown events.

Antigravity Prompt for This Phase

Build a WebSocket gateway (can live in `apps/api` or a new `apps/ws-gateway`) authenticated via the existing JWT, with channels scoped per project+session. After `apps/worker` successfully writes a batch, publish a notification (session id + event ids) to a Redis pub/sub channel named per session. The gateway subscribes to Redis pub/sub and fans out matching messages to connected dashboard WS clients. Build `apps/dashboard's /sessions/[id]/live` page: on mount, opens a WS connection, appends incoming events into the same tab components built in Phase 7, shows a 'live' badge, and cleanly closes the socket on unmount. Implement client-side exponential-backoff reconnect with a guard against duplicate subscriptions on the same channel after a reconnect.

Phase 9 — Retention Policy Engine (~1 day)

Objective

Prove the platform can responsibly delete data, not just collect it — a real product decision, not an afterthought, and a strong interview talking point on data-handling maturity.

Tasks

- Add retentionDays to Project (default 30) and a settings UI on the dashboard to change it.
- Build a BullMQ repeatable job (daily) on the retention queue: find sessions older than the project's retentionDays, cascade-delete their events, write one RetentionAudit row per sweep (count deleted, timestamp, projectId).
- Build a manual 'delete now' action in the dashboard that calls the exact same deletion code path as the scheduled sweep — one deletion implementation, not two.
- Surface the last sweep result (count deleted, when) in the project settings page.

Acceptance Test

Seeding a session with a startedAt older than the project's retentionDays and running the sweep job manually deletes that session and its events and writes a RetentionAudit row; a session inside the retention window is untouched by the same sweep; the manual delete-now button in the dashboard produces an identical RetentionAudit row shape.

Antigravity Prompt for This Phase

Add a retentionDays field (default 30) to the Project model with a migration. Build a BullMQ repeatable daily job on the retention queue that finds sessions with startedAt older than the owning project's retentionDays, cascade-deletes their events, and writes one RetentionAudit row (projectId, deletedCount, sweptAt) per run. Add a project settings page section in the dashboard showing the current retentionDays (editable) and the result of the last sweep. Add a manual 'Delete expired data now' button that calls the same deletion function used by the scheduled job, not a duplicate implementation. Write a test seeding one expired and one non-expired session and asserting only the expired one is removed.

Phase 10 — The AI Reasoning Layer — Summarizer, Anomaly Detection, NL Query, Fix Suggestions (~4 days, largest single phase — deliberate)

Objective

This is the equivalent of Concord's Playground phase: the feature that makes a visitor go from "reading a demo" to "this thing actually understands my problem." Build it as a real subsystem with its own redaction and caching discipline — not a chat box bolted onto the dashboard.

10.1 Session Summarizer (build first)

On session end, the worker enqueues an ai-jobs task that builds a compact structured digest (event counts, error list, slow requests, navigation path — never raw bodies) and sends it to the Claude API for a 3-4 sentence plain-English summary, stored on Session.aiSummary and pushed to the dashboard via the Phase 8 WebSocket channel when ready.

10.2 Anomaly & Pattern Detector

A scheduled ai-jobs task scans recent sessions per project for patterns: duplicate-request storms, error-rate spikes against the project's own baseline, navigation loops, memory/perf-trend flags. The AI proposes candidate findings; deterministic code assigns severity and writes rows to the Finding table — keeps results explainable, not a black box.

10.3 Natural-Language Session Query

A dashboard search bar: a plain-English question is translated by the AI service into a constrained, schema-validated filter object against an allow-listed set of queryable fields — never raw SQL generated by the model. Out-of-scope or ambiguous queries are rejected with a clear message, not silently guessed at.

10.4 Error Triage & Fix Suggestions

For captured JS errors, classify likely cause (network / state / rendering / native bridge) and suggest a fix direction with a confidence label, cached per error signature so the same recurring error isn't re-analyzed (and re-billed) every time it's viewed. Always presented in the UI as a suggestion, never as an authoritative diagnosis.

Tasks

- Build apps/worker/src/ai/ (or a separate apps/ai-service): a Claude API client wrapper whose input type cannot contain raw request bodies/headers — enforce this at the type level, not just by convention.
- Implement buildSessionDigest(sessionId), the summarizer job, and the WebSocket push of the finished summary.
- Implement the anomaly-detector scheduled job and the Finding table writer.
- Implement the NL-query endpoint with an explicit allow-listed field schema and a rejection path for out-of-scope questions.
- Implement the fix-suggestion job with per-error-signature caching to bound AI spend.
- Add an OpenTelemetry metric for AI call latency and (approximate) token cost, tracked like any other backend dependency.

Acceptance Test

Ending a demo session with at least one failed request produces a plain-English summary visible on the session page within a reasonable delay, pushed live if the tab is open. A seeded burst of duplicate requests produces a Finding row with the correct type. A natural-language query like "sessions with failed requests on Android" returns correctly filtered results and a query outside the allow-listed schema is rejected with a clear message, not a guessed SQL query. A unit test proves the AI client wrapper's input type cannot carry raw headers or bodies.

Antigravity Prompt for This Phase

Build the AI reasoning layer per Section 10 of this plan. First, a Claude API client wrapper in `apps/worker/src/ai/client.ts` whose accepted input type structurally excludes raw request headers/bodies (enforce via TypeScript types, not just runtime checks, and add a unit test attempting to pass a raw body and asserting a type/compile error or runtime rejection). Then implement: (1) `buildSessionDigest(sessionId)` and a summarizer BullMQ job on the `ai-jobs` queue that stores the result on `Session.aiSummary` and publishes it over the Phase 8 WebSocket channel, (2) a scheduled anomaly-detector job that proposes findings (duplicate-request storms, error-rate spikes vs. project baseline, navigation loops) and writes them to a Finding table with a deterministic severity assignment, (3) a POST `/v1/projects/:id/query` endpoint that translates a natural-language question into a validated filter object against an explicit allow-list of Session/Event fields, rejecting anything outside that allow-list with a structured error, (4) a fix-suggestion job for captured errors, caching results per error signature (hash of message+stack) to avoid redundant AI calls. Add an OpenTelemetry histogram for AI call latency. Build the dashboard-side AI insights panel on the session page (summary card, findings list, fix-suggestion badges on errors) and the NL search bar.

Phase 11 — Security Hardening & Redaction Audit (~2 days)

Objective

A dedicated pass, not an assumption — verify every redaction and authorization claim made in earlier phases actually holds under adversarial testing.

Tasks

- Audit every ingestion and AI code path for a case where a raw body/header could leak (grep for payload access outside the `redact()/digest-builder` functions).
- Payload size caps and event-count-per-batch caps enforced at the API boundary, not just documented.
- Confirm Authorization headers and passwords are never written to application logs (grep logs after a full local run).
- Cross-project authorization tests across every route, not just ingestion (dashboard routes too).
- API key rotation flow: generate a new key, old key stops working, no downtime for the SDK if rotated correctly (document the expected client behavior on rotation).

Acceptance Test

A full grep of the codebase for raw payload access outside approved redaction/digest functions returns nothing. An automated test suite attempting cross-project access on every authenticated route (dashboard and API) is rejected. A local run's logs contain zero instances of a known test Authorization header value planted for this test.

Antigravity Prompt for This Phase

Run a security audit pass per Section 11 of this plan: grep the codebase for any place a raw event payload, request body, or header object is accessed outside the Phase 1 `redact()` function or the Phase 10 digest builder, and fix any found. Add explicit payload-size and event-count caps at the API boundary with tests proving oversized payloads are rejected with 400/413, not silently truncated. Write a test suite that attempts cross-project access on every authenticated route (both REST API and dashboard) and asserts 403 on each. Add a test that plants a known fake Authorization header value through a full local session and asserts it never appears in any log output. Document the expected SDK behavior when an API key is rotated.

Phase 12 — Testing & CI/CD (~3 days)

Objective

Make every acceptance test from Phases 0-11 automated and running on every PR, not something you remember to check manually.

Tasks

- SDK unit tests: ring buffer, event bus, redaction, collectors (mocked fetch/console/navigation).
- API/worker integration tests: auth (both paths), validation, rate limits, retry/dead-letter, idempotency on duplicate events.
- AI service tests: redaction-guarantee test, NL-query allow-list enforcement, fix-suggestion caching.
- Dashboard component tests (RNTL-equivalent for web) for session list/detail and the AI insights panel.

- One Playwright E2E test covering the full path: create project -> install demo -> send events -> see live session -> see AI summary appear.
- GitHub Actions pipeline: lint -> typecheck -> unit -> integration -> build -> E2E smoke, on every PR; a separate scheduled nightly run of the full E2E suite.

Acceptance Test

All test suites pass locally and in CI. The GitHub Actions pipeline blocks a PR from merging if any stage fails. The E2E test reliably passes on a clean CI runner, not just locally.

Antigravity Prompt for This Phase

Consolidate and automate testing per Section 12 of this plan. Ensure Vitest unit tests exist and pass for the SDK (ring buffer, event bus, redaction, mocked collectors) and for API/worker (auth for both key and JWT paths, Zod validation, rate limiting, BullMQ retry/dead-letter behavior, idempotency on a duplicate-event replay). Add AI-service tests for the redaction-guarantee type/runtime check, the NL-query allow-list rejection path, and fix-suggestion cache-hit behavior. Add dashboard component tests for the sessions list, session detail tabs, and the AI insights panel. Write one Playwright E2E test: create a project, install/run the demo RN app pointed at a local API, send events, open the live session page, and assert the AI summary eventually appears. Build a GitHub Actions workflow running lint, typecheck, unit, integration, build, and the E2E smoke test on every PR, plus a separate nightly scheduled full E2E run.

Phase 13 — Packaging, Deployment & Launch (~3 days)

Objective

Ship it — publish the SDK, deploy every service, and produce the README/demo assets that actually get a recruiter to look twice.

Tasks

- Finalize the public SDK API surface, add TypeScript doc comments, set up semantic versioning and a CHANGELOG.
- Publish @rn-studio/sdk to npm (public access).
- Deploy: dashboard to Vercel, API+worker+WS gateway to Railway/Render (Docker), Postgres on Neon, Redis on Upstash, object storage on Cloudflare R2.
- Run prisma migrate deploy as a release step in CI against production; verify BullMQ repeatable jobs (retention sweep, anomaly scan) are actually scheduled in production, not just locally.
- Load-test ingestion locally (k6 or autocannon) and record real events/sec and p95 ingestion latency for the README and resume — never invent these numbers.
- Record a 30-45s demo screen capture: drag the bubble, open the panel, show a live session on the dashboard, show the AI summary appearing.
- Write the top-level README: one-line pitch, GIF, install command, 5-line quickstart, architecture diagram, measured performance numbers, security/privacy notes, explicit out-of-scope list, roadmap, ADR links.

Acceptance Test

A stranger can clone the repo, follow the README, and run the full stack locally via Docker Compose. The hosted dashboard and API are reachable with zero install. The SDK installs from npm and connects to the hosted demo cluster/environment with the documented 5-line quickstart, unmodified. The README GIF matches what actually happens when followed step by step.

Antigravity Prompt for This Phase

Prepare the v1.0 release per Section 13 of this plan. Finalize @rn-studio/sdk's public exports and TypeScript doc comments, set up semantic-release or manual semver + CHANGELOG.md, and publish to npm. Write a docker-compose.yml for full local parity (Postgres, Redis, API, worker, WS gateway, dashboard). Write GitHub Actions release jobs deploying the dashboard to Vercel and the API/worker/WS gateway as a Docker image to Railway or Render on merge to main, running `prisma migrate deploy` against production as a release step. Write a k6 or autocannon load-test script against the ingestion endpoint and output events/sec and p95 latency to a file the README can reference. Write the top-level README.md with: a one-line pitch, an install command, a 5-line SDK quickstart, an architecture diagram (ASCII is fine), the measured performance numbers from the load test, a security/privacy section describing the redaction guarantees from Phase 1/11, an explicit out-of-scope list (crash symbolication, video replay, arbitrary JS execution, production telemetry by default), and a roadmap section linking the docs/ADR folder.

Stretch Phases (Optional, Only After Phase 13 Ships)

Do not start either of these until the v1.0 launch checklist below is fully checked. Both are genuine differentiators if you have runway left, but a shipped, polished v1.0 beats an unfinished v1.5 every time in front of a recruiter.

Phase 14 — Native Module Milestone (Turbo Native Module) (~3-4 days, stretch)

Objective

Demonstrate React Native engineering below the JS layer without rewriting the debugger in native code.

Tasks

- Pick one small native capability: a native memory-pressure signal, an app lifecycle event, or a platform-specific performance metric.
- Define a typed JS/TS spec and use Codegen to generate the native interface (per React Native's current Turbo Native Module documentation).
- Implement the native side on iOS and/or Android, surface the metric as a fifth Device-tab data point.

Acceptance Test

The native signal appears correctly in both the in-app panel and the dashboard's Device tab, sourced from a real native call, not a JS-side approximation.

Phase 15 — Session Replay / Performance Timeline (~1 week, stretch)

Objective

The highest-value stretch feature after native modules — reconstructing a session's event sequence on a scrubbable timeline with JS/UI thread performance sampling.

Tasks

- Extend the event model with monotonic timestamps precise enough for timeline reconstruction.
- Build a timeline scrubber component on the dashboard session view driven purely from stored events (no new capture mechanism).
- Add basic JS-thread frame-drop sampling to the SDK, surfaced as an overlay on the timeline.

Acceptance Test

Scrubbing the timeline on a real captured session correctly re-orders and re-displays network, console, and navigation events matching their original sequence, and the frame-drop overlay lines up with a manually-induced slow render in the demo app.

16. Testing & Correctness Matrix (Full Project, Phases 0-13)

Concern	Test type	Introduced in
Ring buffer eviction, event bus ordering, redaction correctness	Unit	Phase 1
Panel renders/switches tabs without crashing host app	Component	Phase 2
Collectors capture and redact real events correctly	Unit + manual	Phase 3
Cross-project auth isolation (API + dashboard)	Integration	Phase 4, re-verified Phase 11
Queue retry, dead-letter, idempotency on duplicate events	Integration	Phase 5
Index usage on session/event queries at scale	EXPLAIN-based	Phase 6
Session list/detail correctness and pagination	Component + integration	Phase 7
Live event delivery latency, reconnect without duplication	Integration, manual timing	Phase 8
Retention sweep deletes only expired data, audit logged	Integration	Phase 9
AI redaction guarantee (no raw bodies sent)	Unit (type-level + runtime)	Phase 10
NL-query allow-list enforcement, rejects out-of-scope questions	Integration	Phase 10
Fix-suggestion caching avoids redundant AI spend	Unit	Phase 10
Payload/log leakage of secrets under adversarial test	Security audit + automated grep test	Phase 11
Full pipeline E2E: install -> send -> live view -> AI summary	Playwright E2E	Phase 12
Production migration + scheduled jobs actually run	Manual + smoke test post-deploy	Phase 13

17. Total Time Estimate From Today (Through Phase 13)

Phase	Status	Estimate
0. Project definition & setup	Not started	~1 day
1. SDK core (event bus, buffer, types)	Not started	~2 days
2. Draggable panel shell	Not started	~2 days
3. Collectors	Not started	~3 days
4. Cloud ingestion + multi-tenant auth	Not started	~3 days
5. Redis/BullMQ pipeline	Not started	~2 days
6. PostgreSQL schema	Not started	~2 days
7. Dashboard core	Not started	~3 days

Phase	Status	Estimate
8. WebSocket live updates	Not started	~2 days
9. Retention policy engine	Not started	~1 day
10. AI reasoning layer	Not started	~4 days
11. Security hardening	Not started	~2 days
12. Testing & CI/CD	Not started	~3 days
13. Packaging, deployment, launch	Not started	~3 days
Subtotal through Phase 13 (v1.0, must-ship set)		~33 days (~4.5-5 weeks full-time)
14. Native module (stretch)	Optional	~3-4 days
15. Session replay / perf timeline (stretch)	Optional	~1 week

A good public v0.5 checkpoint is right after Phase 6 (real ingestion pipeline, real auth, real database — already a legitimate, demoable backend systems project on its own, before the dashboard exists). A strong v0.8 checkpoint is right after Phase 9 (functionally complete observability platform with live updates and retention, before the AI layer exists). Phase 10 is what makes it distinctive. Phase 13 is what makes it public.

18. Sequencing Rule

Do not begin a phase until the previous phase's acceptance test passes. Within Phase 10 specifically, do not begin the fix-suggestion feature (10.4) until the summarizer (10.1) is fully working and pushed live via WebSocket — the later AI features reuse the same client wrapper and redaction guarantee, and debugging that wrapper is much easier with only one feature depending on it. If time runs short anywhere in Phases 10-13, cut Phase 10's NL query (10.3) before cutting the summarizer or anomaly detector — the summarizer is the single highest-value AI feature for a first-impression demo.

19. Launch Checklist

- MIT or Apache-2.0 license, CI badge, GitHub Actions running lint/unit/integration/E2E on every PR.
- docs/ADR/ folder with a decision record for every major choice (monorepo layout, auth split, single-deletion-path retention, AI redaction guarantee, REST-vs-queue boundaries).
- README leads with a GIF of the draggable panel and a live session updating on the dashboard, then the 5-line SDK quickstart.
- README states the deliberate out-of-scope list (crash symbolication, full video replay, arbitrary JS execution, production telemetry by default) as intentional scoping, framed honestly rather than apologetically.
- Real, measured performance numbers in the README and resume (events/sec, p95 ingestion latency, memory overhead) — never invented.
- SDK published on npm with its own README and the 5-line quickstart runnable against the hosted demo, unmodified.
- Hosted, zero-install dashboard + demo environment linked at the top of the README.
- Security/privacy section explicitly describing what is and is not redacted before leaving the device, and what is/isn't sent to the AI provider.
- A 30-45 second demo video/GIF and a short LinkedIn launch post drafted for the day you ship.

Closing note: nothing in this plan is deferred out of the MVP the way a slower timeline might defer it — Redis/BullMQ, multi-tenant auth, WebSockets, and retention are all built as load-bearing parts of the first two weeks, and the AI layer is treated as a real subsystem with its own correctness and security discipline, not a demo gimmick. That combination is what should make this stand out in a 2026 React Native hiring pipeline.